

MeetingMind

What we built and how it works

Deon Menezes (16014223030) · Aishwarya Gawali (16014223007) · Batch B1

Generative AI Laboratory, Experiment 8 (Mini Project) · Code: github.com/DeonMenezes-11/MeetingMind

In one sentence: MeetingMind takes a recording of a meeting and gives back a transcript that knows who said what, the decisions and tasks that were agreed (each one backed by the exact words that prove it), ready-to-send follow-up emails, and a chat in which you can ask questions about the meeting and hear the answer being said.

1. The problem we are solving

In most meetings, decisions are made and tasks are handed out, but the only record is what someone remembered to write down. Afterwards people disagree about who promised what and by when. A speech-to-text model can write a transcript, and a large language model (LLM) can summarise it, but an LLM can also **make things up** - give a task to the wrong person or invent a deadline - and a reader cannot tell which parts are real. A transcript is also text spoken by people, so it can even contain sentences that try to trick the AI. MeetingMind is designed so that every output can be checked against what was actually said.

2. What happens when you upload a meeting

The work is split into eight stages. The teal boxes call an OpenAI model, the grey boxes are ordinary code that prepares the audio and checks the AI's answers, and the blue box is the website people use.

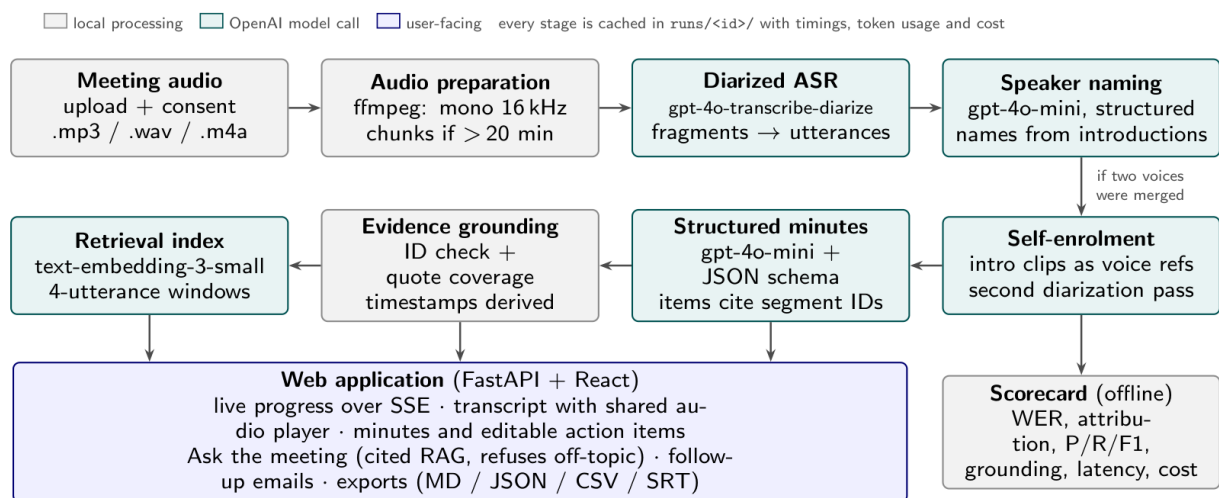


Figure 1 - The MeetingMind pipeline, from audio to the web application.

1. **Prepare the audio** - `ffmpeg` converts any recording to a standard format (mono, 16 kHz) and splits very long meetings into chunks.
2. **Transcribe and separate the speakers** - `gpt-4o-transcribe-diarize` writes down the words and labels each sentence with a speaker (A, B, C...).
3. **Find out who is who** - `gpt-4o-mini` reads the introductions ("Hi, I'm Rohan, the back-end developer") and turns labels into names and roles.
4. **Fix merged voices (self-enrolment)** - if two people were given the same label, each person's own introduction is cut out and used as a voice sample for a second, more accurate pass. This raised correct speaker attribution from 80.5% to 100% in our test.
5. **Write the minutes** - `gpt-4o-mini` produces a summary, topics, decisions, action items (owner, task, due date, priority), open questions and risks. For every item it must quote the exact sentence it came from.
6. **Check the evidence** - our own code verifies each quote against the transcript. Items that cannot be matched are flagged "needs review" and are not emailed.

7. **Build the question-answering index** - the transcript is split into small overlapping pieces and turned into vectors with `text-embedding-3-small`, so questions can find the right part of the meeting (retrieval-augmented generation, RAG).
8. **Export** - minutes (Markdown, JSON), action items (CSV) and subtitles (SRT) are written to disk.

3. Using the website

The pipeline is wrapped in a web application: a Python **FastAPI** server and a **React** website. Every page of a meeting shares one audio player at the bottom of the screen, so clicking any timestamp plays that moment.

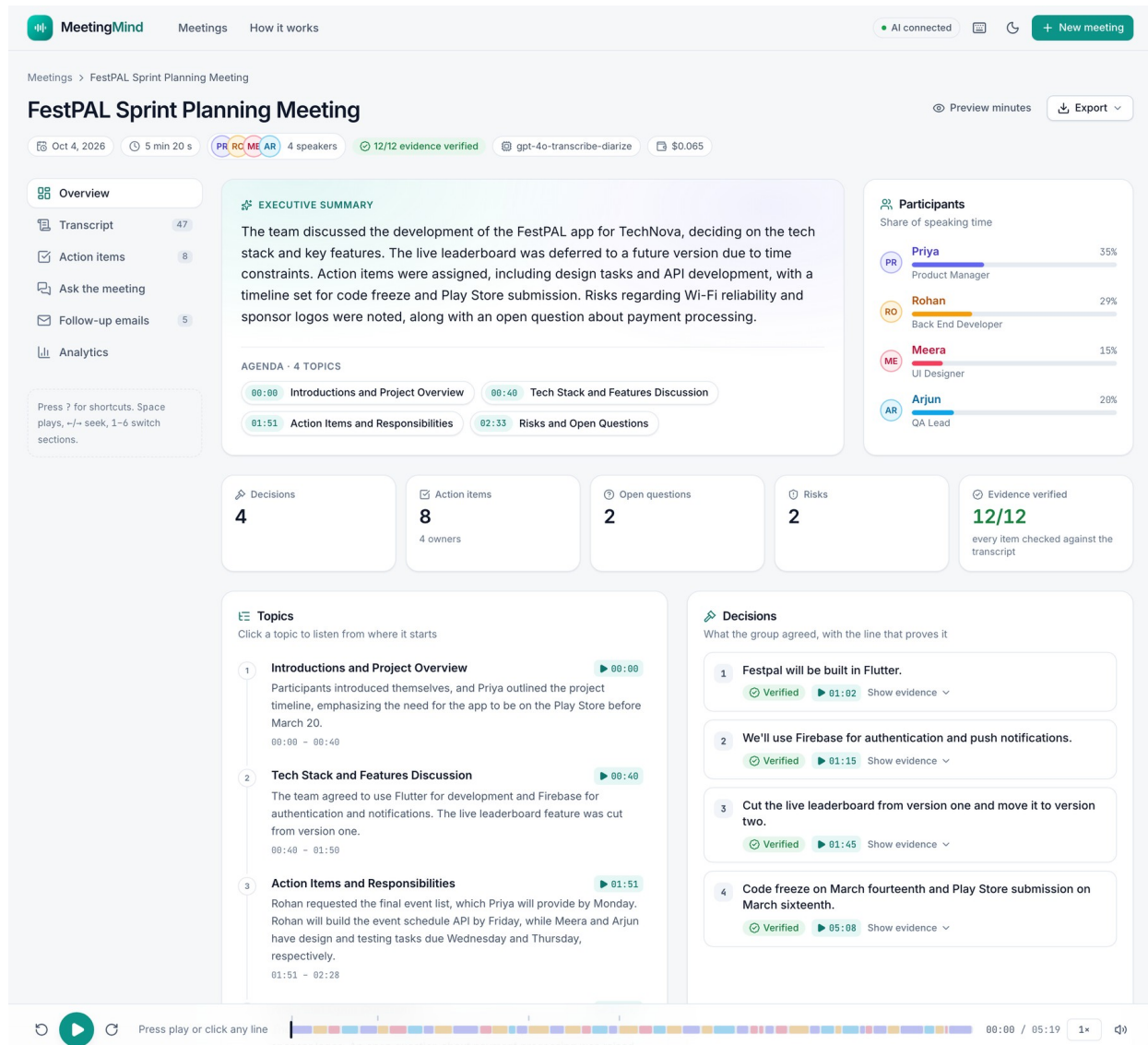


Figure 2 - The meeting workspace (Overview). The coloured bar at the bottom is the audio player; each colour is a speaker.

- **Home** - the list of processed meetings and a button to open the sample meeting.
- **New meeting** - drag in a recording, optionally type the participants' names, confirm everyone agreed to be recorded, and press Process. A live progress page shows each stage as it runs.
- **Overview** - summary, clickable agenda, decisions with the sentence that proves them, open questions and risks.
- **Transcript** - click any line to play it; the line being spoken is highlighted. Search, filter by speaker and rename speakers.
- **Action items** - edit the owner, task, due date and priority; changes are saved and flow into emails and exports.
- **Ask the meeting** - ask in plain English; answers include timestamps you can play, and questions the meeting never covered are politely refused.
- **Follow-up emails** - one email per person plus a team recap, formal or friendly, ready to copy or open in a mail app.

- **Analytics** - who spoke when, share of speaking time, and how long and how much each stage cost.

4. Why the output can be trusted

- **Evidence for every item** - the AI must cite the sentence; code checks it. Unverified items are clearly marked.
- **No guessing in Q&A** - if no part of the meeting is similar enough to the question, the AI is not even asked; if it is asked but the answer is not in the excerpts, it must reply that it was not discussed.
- **Resistant to tricks** - the transcript is treated as data, not instructions. Our test meeting contains a spoken "ignore your previous instructions" line, and it was ignored.
- **Human in the loop and privacy** - people review action items before emails are drafted, nothing is sent automatically, the API key never leaves the server, and recordings require consent.

5. How well it works

We scripted a 5 min 20 s sprint-planning meeting between four people, generated the audio with a text-to-speech model (so we know exactly who said what and when), and also made a copy with background noise. All scores are computed by code, not by an AI judging itself.

Measure	Clean audio	Noisy audio (10 dB)
Word error rate (lower is better)	4.0%	3.5%
Speaking time given to the right person	100%	92.6%
Action items found (precision / recall)	100% / 100%	100% / 87.5%
Decisions found	4 of 4	4 of 4
Items that passed the evidence check	12 of 12	11 of 11
Processing time and cost	about 4 min, \$0.065	about 4 min, \$0.065

6. Technology and how to run it

Part	What we used
AI models (OpenAI API)	gpt-4o-transcribe-diarize, gpt-4o-mini, text-embedding-3-small
Pipeline	Python 3, Pydantic structured outputs, NumPy, ffmpeg
Web application	FastAPI back end (live progress with server-sent events), React + TypeScript + Tailwind CSS
Quality	78 automated tests, evaluation scorecard, Docker image for deployment

Run it locally: put your OpenAI key in a `.env` file, then run `pip install -r requirements.txt` and `python3 server.py --open`. The website opens at <http://localhost:8531>, and the sample meeting is available immediately without any API calls.

Source code and the IEEE-style report: <https://github.com/DeonMenezes-11/MeetingMind>